

SET1 A5. Найти всех простых

Код на C++:

```
// Краткое обоснование корректности кода:  
// Каждое составное число  $N = i * p$ , где  $p$  - наим простый делитель  $N$   
// Для простых чисел:  $\text{min\_pr}[i] == 0 \rightarrow$  число простое, добавляем в primes  
// Для составных чисел: Каждое число  $i * \text{primes}[j]$  помечается ровно один раз -  
// - когда обрабатывается число  $i$  и простое  $p = \text{primes}[j]$ , при этом:  
// 1. Условие  $\text{primes}[j] \leq \text{min\_pr}[i]$  гарантирует, что  $\text{primes}[j]$  будет наименьшим  
делителем для  $i * \text{primes}[j]$   
// 2. Прерывание при  $\text{primes}[j] == \text{min\_pr}[i]$  гарантирует, что каждое число  
обрабатывается только один раз  
// Алгоритм обрабатывает каждое составное число в момент его генерации через  
наименьший простой делитель  
// Это обеспечивает полноту и отсутствие пропусков.
```

```
#include <iostream>
```

```
#include <vector>
```

```
std::vector<int> get_primes(int N) {  
    std::vector<int> primes;  
    std::vector<int> min_pr(N + 1, 0);  
  
    for (int i = 2; i <= N; ++i) {  
        if (min_pr[i] == 0) {  
            min_pr[i] = i;  
            primes.push_back(i);  
        }  
        for (int j = 0; j < (int)primes.size() && primes[j] <= min_pr[i] && i * primes[j] <= N; ++j) {  
            min_pr[i * primes[j]] = primes[j];  
            if (primes[j] == min_pr[i]) {  
                break;  
            }  
        }  
    }  
    return primes;  
}
```

```
int main() {  
    int N;  
    std::cin >> N;  
  
    std::vector<int> primes = get_primes(N);  
  
    for (int prime : primes) {  
        std::cout << prime << " ";  
    }  
    std::cout << "\n";  
  
    return 0;  
}
```

Анализ временной сложности $T(N)$

Подсчет операций:

1. Внешний цикл: `for (int i = 2; i <= N; ++i)` - выполняется $N-1$ раз + каждая итерация: сравнение $i \leq N$, инкремент $++i$
2. Проверка условия: `if (min_pr[i] == 0)` - выполняется $N-1$ раз + операция сравнения
3. Обработка простых чисел (когда `min_pr[i] == 0`):
 - `min_pr[i] = i` выполняется $\pi(N)$ раз (т.к. количество простых $\leq N$)
 - `primes.push_back(i)` - выполняется $\pi(N)$ раз
4. Внутренний цикл: `for (int j = 0; j < primes.size() && ...)`
`min_pr[i * primes[j]] = primes[j]` соответствует уникальному составному числу, количество таких присваиваний = количеству составных чисел $\leq N = N - \pi(N) - 1$
5. Дополнительные проверки во внутреннем цикле выполняются немного больше раз, чем присваивания, но не более чем в константное число раз больше

Введем обозначения:

- K - количество дополнительных проверок ($K = O(N)$)
- c_1, c_2, c_3, c_4 - константы времени на операции
- $\pi(N) \sim N/\ln(N)$ (по теореме о распределении простых чисел)

$$T(N) = c_1 \cdot (N-1) + c_2 \cdot \pi(N) + c_3 \cdot (N - \pi(N) - 1) + c_4 \cdot K = c_1 \cdot N + c_2 \cdot (N/\ln(N)) + c_3 \cdot N + c_4 \cdot O(N) = (c_1 + c_3 + c_4) \cdot N + o(N) = O(N)$$

Доказательство $T(N) = O(N)$

Линейность достигается за счет того, что каждое составное число обрабатывается ровно один раз при установке его наименьшего простого делителя.

$T(N) = O(N)$ если существуют константы $C > 0$ и $N_0 > 0$ такие, что для любого $N > N_0$: $T(N) \leq C \cdot N$

Для начала оценим количество операций: внешний цикл: $\leq N$ итераций, внутренний цикл: каждая итерация с присваиванием соответствует уникальному числу $\leq N$. Общее количество операций $\leq K \cdot N$, где K - константа.

Предположим $C = 10$ (каждая итерация внешнего и внутреннего циклов требует не более 10 элементарных операций), тогда $N_0 = 100$. Получаем, что для любого $N > 100$: $T(N) \leq 10 \cdot N$ - это выполняется, поскольку:

1. Количество обработанных чисел $\leq N$
2. Каждое число обрабатывается константное число раз
3. Вспомогательные операции на управление циклами также $O(N)$

По определению: $T(N) = O(N)$ ч.т.д.